

**Beyond the Budget: Predicting Box Office Performance Using
Production Features and Digital Signals**

Charlotte Thunen and Isabella Fratantonio

DS4420: Machine Learning and Data Mining 2

Dr. Gerber

13 March 2026

I. Introduction and Literature Review

In 2025, the global film industry generated over \$33 billion in box office revenue and the US domestic film industry generated a projected \$9.8 billion in box office revenue¹. As these numbers continue to grow towards surpassing pre-pandemic levels, predicting whether a given movie will succeed or fail at the box office remains notoriously difficult. Predicted box office performance is a highly sought after metric by studios, distributors and investors as it can inform decisions around production budget, casting, release timing, and distribution strategy. Historically, box office forecasting relied on simple comparisons to past films and relying on gross revenue². More recently, researchers have turned to machine learning and statistical modeling to capture the complex, nonlinear relationships among the many factors that influence a film's commercial performance.

The intent of our project was to look at a combination of production features and social discourse around movies, to see if a combined approach would enhance box office predictions. We investigate the prediction of box office revenue using two complementary modeling approaches: a Bayesian linear regression model implemented fully by hand in R, and a Multilayer Perceptron (MLP) implemented in Python using NumPy with sklearn for preprocessing. A central question motivating our analysis is whether pre-release digital signals, specifically Google Trends search interest, improve predictive accuracy beyond what typical production-level features such as budget, genre, cast popularity, and release timing can achieve. Since the Twitter API is no longer available for free use, we were limited to just using Google Trends, though it is a less comprehensive measure of social discourse than we had initially planned. By training each model on both a traditional feature set, and an enriched feature set that includes Google Trends data, we aim to quantify the marginal value of online audience interest data for box office revenue forecasting.

A. *Machine Learning Approaches to Box Office Prediction*

Box office prediction is an increasingly active area of machine learning research, with studies exploring a range of modeling strategies. Lee, KC, and Choeh explored ensemble methods including k-nearest neighbors, decision trees, and linear regression which outperformed

¹ AMW World Group. 2026. "Film & Box Office Statistics 2026 | Revenue & Streaming Data." AMW World Group. <https://amworldgroup.com/statistics/film-box-office-statistics>.

² Hannan, Brian. 2022. "When Box Office Stats Were Born - 1969." The Magnificent60s. <https://themagnificent60s.com/2022/01/12/when-box-office-stats-were-born/>.

individual models by better capturing non-linear trends³. Li and Liu proposed a method for predicting nationwide box office revenue using economic factors comparing eight methods which included support vector machines⁴. Their work demonstrated that even economic modeling techniques carry meaningful predictive signals for box office revenue, their SVM-based method using GDP as a predictor achieved the strongest performance.

B. Social Media and Digital Signals for Revenue Prediction

A similar line of research has explored whether user-generated online activity can serve as a predictive signal for box office outcomes. Bogaert et al. provided a cross-platform comparison evaluating Facebook and Twitter data alongside traditional movie characteristics⁵. They found that social media signals increased predictive performance beyond traditional features, though the magnitude of improvement varied by platform. Their study reinforced the notion that digital signals capture dimensions of audience engagement, and was especially useful in predicting audience reception. Their work captured organic audience reactions and used it in complement to traditional production features to predict the rating and overall sentiment related to movies.

C. Research Gap and Our Contribution

Past research has shown that both production-level features and digital signals can help predict box office performance, however, most of these studies use pre-built ML packages and give point estimates rather than capture how uncertain those predictions are. Bayesian methods haven't gotten much attention in this space, even though they're a natural fit.

Our project aims to fill that gap. We built a manually implemented Bayesian linear regression model which gives us full posterior distributions over predicted revenue instead of just one estimate. We compare that against an MLP in Python to see how a probabilistic approach stacks up against a deterministic, nonlinear one. We test both models on traditional features alone and then on an enriched feature set that adds in Google Trends search interest. This lets us directly measure whether digital signals actually improve accuracy across both modeling approaches.

³ Lee, Sangjae, Bikash KC, and Joon Yeon Choeh. 2020. "Comparing performance of ensemble methods in predicting movie box office revenue." *Heliyon* 6, no. 6 (June). <https://doi.org/10.1016/j.heliyon.2020.e04260>.

⁴ Li, Dawei, and Zhi-Ping Liu. 2022. "Predicting Box-Office Markets with Machine Learning Methods." *Entropy* 24, no. 5 (May): 711. 10.3390/e24050711.

⁵ Bogaert, Matthias, Michel Ballings, Dirk Van den Poel, and Asil Oztekin. 2021. "Box office sales and social media: A cross-platform comparison of predictive ability and mechanisms." *Elsevier* 147 (August). <https://doi.org/10.1016/j.dss.2021.113517>.

II. ML Methodology and Data Collection

A. Data Collection and Curation

Our raw data comes from the TMDb 5000 Movie Dataset on Kaggle, which contains metadata for 5,000 movies including budget, revenue, cast, crew, genres, keywords, and production companies. We merged the movies and credits tables on movie ID, and applied several cleaning steps such as removing movies with no reported budget or revenue, filtering to post-2000 releases, and excluding movies with invalid runtimes. This allowed us to narrow our scope down to reflect the modern box-office landscape and film industry, leaving us with 2,276 movies for training and testing our models.

We used 25 production level features from the raw metadata. Continuous features included log-transformed budget, runtime, popularity score, vote average, and vote count. We created count features for the number of genres (one-hot encoded), cast members, and production companies associated with each film. Binary indicators capture whether a movie was released by a major studio (including Warner Bros., Universal, Paramount, Disney, Columbia, 20th Century Fox, New Line Cinema, Lionsgate, DreamWorks, or Sony), seasonality of when the movie was released (summer months or holiday months), whether it is part of a franchise, and whether the original language is English. Our target variable is the log-transformed box office revenue.

For our enriched feature set, we collected Google Trends using the pytrends library. For each movie, we queried the average search interest for the film's title in the United States during the 28 days prior to its release date. This pre-release search volume serves as a proxy for audience anticipation.

B. Multilayer Perceptron (MLP)

Our first model is a single hidden layer MNLP implemented in Python. We use sklearn's MinMaxScaler for feature normalization and train test split for an 80/20 train/test partition, but the network architecture, forward pass, gradient computation, and training loop are all written manually using NumPy.

The network takes the 25 scaled features and a bias term as input, passes them through a hidden layer of 32 units using the ReLU activation function, and produces a single linear output predicting the log-revenue.

Training proceeds by full-batch gradient descent over 500 epochs with a learning rate of $\eta = 0.001$, and we derive the gradients analytically. For the output layer weights W_2 , the gradient

is straightforward: the error signal multiplied by the hidden activations. For the hidden layer weights W_1 , we apply the chain rule using the Heaviside step function as the ReLU derivative and compute the outer product via the Kronecker product of the input vector and the backpropagated error. Our manual derivation follows the mathematical framework covered in class lectures during the MLP unit.

C. Bayesian Linear Regression

Our second model is a Bayesian linear regression implemented entirely by hand in R, using no external packages. Rather than producing a single point estimate, this model yields full posterior distributions over the regression coefficients and the noise variance, giving us an uncertainty quantification.

We place a Gaussian prior on the coefficient vector $\beta \sim N(0, \tau^2 I)$ with $\tau^2 = 100$, which is weakly informative and allows the data to dominate. The noise variance receives an Inverse-Gamma prior $\sigma^2 \sim IG(a_0 = 2, b_0 = 1)$. The posterior is sampled using the Metropolis-Hastings algorithm with 10,000 MCMC iterations and a 2,000 iteration burn in. Proposals for β are drawn from a symmetric Gaussian centered on the current value. For σ^2 , we propose in log-space and apply a Jacobian correction to maintain detailed balance, which ensures proposals remain positive and improves mixing. After the burn-in, we thin the chain by keeping every 10th sample to reduce autocorrelation.

We assess convergence using trace plots and autocorrelation function (ACF) plots for both the coefficients and the variance parameter. The chain is initialized at the ridge-regression solution to start near the posterior mode. Posterior predictive distributions are constructed by averaging predictions across the thinned posterior samples, and 95% credible intervals are generated by drawing from the predictive distribution at each posterior sample.

III. ML Results and Interpretation

A. MLP Results

The MLP was first trained for 500 epochs on the traditional production-level feature set. Figure 1 shows the training loss curve, which demonstrates clear convergence: MSE drops sharply from over 300 in the first few epochs to under 10 by epoch 100, it then continues to decrease gradually, reaching a final training MSE of 3.49 by epoch 500. The steep initial descent followed by a long plateau is a characteristic of gradient descent on neural networks with ReLU activations.

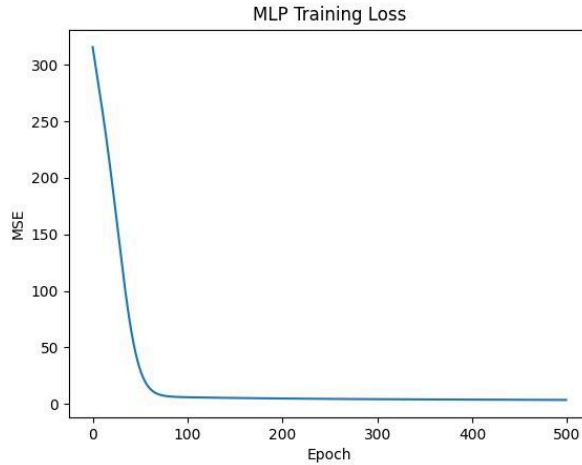


Figure 1: MLP Training Loss (MSE) over 500 epochs, showing convergence

On the test set, the traditional MLP achieved an RMSE of 1.8116, MAE of 1.239, and R^2 of 0.2240. While the model clearly learns a meaningful signal from the production features, explaining only about 22% of the variance in log revenue suggests that a single hidden layer architecture with 32 nodes may not have sufficient capacity to capture the complex nonlinear interactions present in the data.

When we added Google Trends pre-release search interest as a feature, creating our enriched dataset, the MLP's performance actually declined: RMSE increased to 1.8803, MAE to 1.3151, and R^2 dropped to 0.1640. Table 1 summarizes the comparison of the model's performance on the traditional vs. the enriched dataset.

Table 1: MLP test set performance, traditional vs. enriched features

Metric	Traditional	Enriched
RMSE	1.8116	1.8803
MAE	1.2391	1.3151
R^2	0.2240	0.1640

These results suggests that the Google Trends signal, at least as a raw 28-day average, introduces noise that the MLP overfits to rather than extracting useful information. With only 2,276 training examples and a relatively shallow architecture, the network may lack the regularization needed to ignore a weak or noisy feature.

B. Bayesian Linear Regression Results

The Bayesian model was run for 10,000 MCMC iterations with a 2,000 iteration burn-in. Trace plots for the bias coefficient, the log-budget coefficient, and σ^2 all showed stable mixing after burn-in, and ACF plots after thinning confirmed that autocorrelation was effectively eliminated.

Figure 2 shows the predicted vs. actual log revenue for the test set using the traditional feature set. Points cluster along the identity line (shown in red), particularly for higher-revenue films in the log revenue range of 15-20, where the majority of the data falls. There is more scatter at lower revenue values, which is expected given that low-revenue films are both rarer in the dataset and likely driven by idiosyncratic factors not captured by our production metadata.

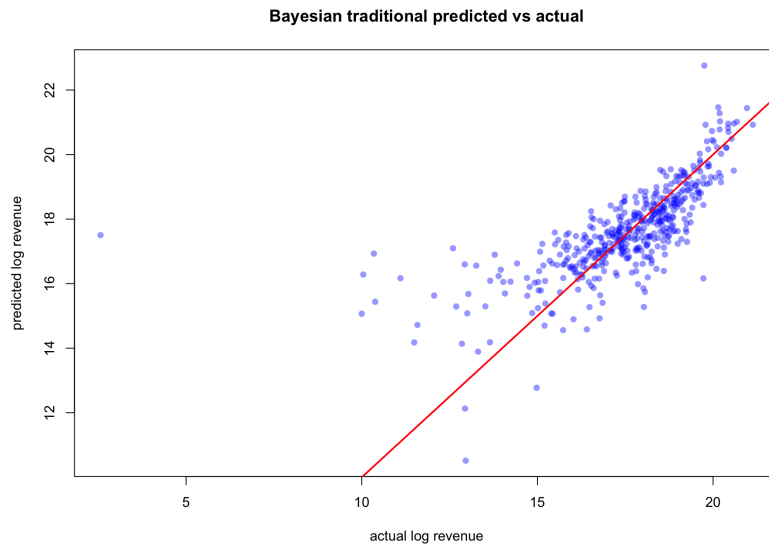


Figure 2: Bayesian model predicted vs. actual log revenue (traditional features).

The traditional Bayesian model achieved an RMSE of 1.3517, MAE of 0.8196, and R^2 of 0.5246 on the test set. This substantially outperforms the MLP across all three metrics, explaining over 52% of the variance in log revenue compared to the MLP’s 22%. The strongest posterior predictors were log_budget, popularity, and vote_count, which had the largest posterior means and 95% credible intervals that did not include zero.

When Google Trends was added as an additional feature, the Bayesian model showed a marginal improvement: RMSE decreased to 1.3493, MAE to 0.8179, and R^2 increased to 0.5263. Table 2 summarizes the comparison.

Table 2: Bayesian test set performance, traditional vs. enriched features

Metric	Traditional	Enriched
RMSE	1.3517	1.3493
MAE	0.8196	0.8179
R ²	0.5246	0.5233

While the improvement from Google Trends is small, the fact that it is positive, rather than negative as we saw with the MLP, is notable. The bayesian linear model, by nature of its simpler functional form and built-in regularization through the prior, appears better able to absorb a weak signal without overfitting to its noise.

C. Model Comparison

The Bayesian model ($R^2 = 0.525$) substantially outperformed the MLP ($R^2 = 0.224$) on the traditional feature set. This result may seem counterintuitive—a nonlinear model performing worse than a linear one—but there are several plausible explanations. First, with 2,276 training examples and 25 features, the dataset may not be large enough for the MLP’s 32-unit hidden layer to learn robust nonlinear representations without overfitting. Second, our MLP uses no regularization (no dropout, no weight decay), whereas the Bayesian model’s Gaussian prior on β implicitly acts as L2 regularization. Third, full-batch gradient descent with a fixed learning rate may not reach the global optimum, particularly for a non-convex loss surface.

Google Trends had opposite effects across the two models: a marginal improvement for the Bayesian model (R^2 0.5246 $\rightarrow$ 0.5263) but a decline for the MLP (R^2 0.224 $\rightarrow$ 0.164). This suggests that the linear model better absorbs a weak signal while the MLP overfits to its noise. The Bayesian model’s results indicate that digital signals have a slight positive impact on predictive accuracy, warranting further exploration of stronger digital signals.

A key advantage of the Bayesian approach beyond raw accuracy is its uncertainty quantification. The posterior credible intervals provide a calibrated measure of how confident the model is in each prediction, which has practical value for studios and investors making financial decisions based on revenue forecasts. The 95% credible interval coverage on the test set provides a check that the model’s uncertainty estimates are well-calibrated.

IV. Conclusions and Future Work

In this project, we built and compared two complementary approaches for predicting movie box office revenue: a Bayesian linear regression model implemented fully by hand in R, and a multilayer perceptron implemented in Python with NumPy. Both models were trained on a curated set of 25 production metadata features derived from the TMDb 5000 dataset, and then retrained with Google Trends pre-release search interest as an additional feature.

The Bayesian model substantially outperformed the MLP (R^2 of 0.525 vs. 0.224), demonstrating that for this dataset and feature set, a well-specified linear model with principled uncertainty quantification beats a shallow, unregularized neural network. Budget, popularity, and vote count emerged as the strongest predictors across both approaches. Google Trends had a small positive effect on the Bayesian model but hurt the MLP, suggesting that the linear model is more robust to weak or noisy features.

There are several assumptions and limitations worth noting. The Bayesian linear regression assumes a linear relationship between features and log revenue, and normally distributed errors. While log-transforming the revenue helps with the normality assumption, the relationship may still contain nonlinear patterns that the linear model cannot capture. The MLP in principle can model these nonlinearities, but our single-hidden-layer architecture with no regularization appears insufficient for the task at the scale of our dataset. Additionally, both models treat the features as independent, when in practice features like budget and studio are likely correlated.

If we were to continue working on this problem, several directions would be promising. First, incorporating Twitter discourse volume as an additional digital signal would complement the Google Trends data; Bogaert et al. demonstrated that cross-platform social media signals improve predictions, and if the Twitter API becomes accessible again, it would provide valuable insight into social discourse trends. Second, extending the MLP to deeper architectures with regularization techniques such as dropout and L2 weight decay would likely improve its generalization. Third, exploring Bayesian neural networks or Gaussian process regression could combine the MLP's ability to model nonlinear effects with the Bayesian framework's uncertainty quantification. Finally, more sophisticated feature engineering for Google Trends—such as using the trend slope or peak interest rather than the raw 28-day mean—could yield a stronger signal that both models can leverage.

V. Appendix: Implementation

Github Repository: https://github.com/charthunen/DS4420_Final_Proj

A. MLP Training Implementation (Python)

```
21 Phi = data[feature_cols].to_numpy()
22 y = data['log_revenue'].to_numpy()
23
24 # scale
25
26 scaler = MinMaxScaler()
27 Phi_scale = scaler.fit_transform(Phi)
28
29 # add bias column
30 n = Phi_scale.shape[0]
31 Phi_scale = np.column_stack([np.ones(n), Phi_scale])
32
33 # train/test split
34
35 [Phi_train, Phi_test, y_train, y_test] = train_test_split(*arrays: Phi_scale, y, test_size=0.2)
36
37 # MLP setup
38
39 X = Phi_train
40 y_tr = y_train
41
42 p = X.shape[1]
43 q = 32
44 eta = 0.001
45 epochs = 500

```

```
21 Phi = data[feature_cols].to_numpy()
22 y = data['log_revenue'].to_numpy()
23
24 # scale
25
26 scaler = MinMaxScaler()
27 Phi_scale = scaler.fit_transform(Phi)
28
29 # add bias column
30 n = Phi_scale.shape[0]
31 Phi_scale = np.column_stack([np.ones(n), Phi_scale])
32
33 # train/test split
34
35 [Phi_train, Phi_test, y_train, y_test] = train_test_split(*arrays: Phi_scale, y, test_size=0.2)
36
37 # MLP setup
38
39 X = Phi_train
40 y_tr = y_train
41
42 p = X.shape[1]
43 q = 32
44 eta = 0.001
45 epochs = 500

```

```
47 # initialize weights
48 W1 = np.random.randn(p, q) * np.sqrt(2/p)
49 W2 = np.random.randn(q, 1) * np.sqrt(2/q)
50
51 # relu activation
52 def relu(z): 6 usages  ↳ bellafratantonio
53     return np.maximum(0, z)
54
55 # prediction function
56 def f(x):  ↳ bellafratantonio
57     h = relu(W1.T.dot(x))
58     return W2.T.dot(h)
59
60 # training with gradient descent
61 errors = []
62 n = X.shape[0]
63
64 for epoch in range(epochs):
65     # update W2
66     dW2 = 0
67     for i in range(n):
68         x = np.reshape(X[i], newshape: (p, 1))
69         h = relu(W1.T.dot(x))
70         dW2 += (1/n) * (f(x) - y_tr[i]) * h
71
72     W2 = W2 - eta * dW2
```

```

74     # update W1
75     dW1 = 0
76     for i in range(n):
77         x = np.reshape(X[i], newshape: (p, 1))
78         h = relu(W1.T.dot(x))
79         mat1 = np.heaviside(h, 0) # relu derivative
80         dW1 += (1/n) * (f(x) - y_tr[i]) * np.kron(x, (W2 * mat1).T)
81
82     W1 = W1 - eta * dW1
83
84     # calculate MSE
85     loss = 0
86     for i in range(n):
87         x = np.reshape(X[i], newshape: (p, 1))
88         pred = f(x)[0, 0]
89         loss += (pred - y_tr[i]) ** 2
90     errors.append(loss / n)
91
92     if (epoch + 1) % 100 == 0:
93         print(f"Epoch {epoch+1}/{epochs}, MSE: {errors[-1]:.4f}")
94
95 # plot convergence
96
97 plt.plot(*args: range(epochs), errors)
98 plt.xlabel('Epoch')
99 plt.ylabel('MSE')
100 plt.title('MLP Training Loss')
101 plt.show()
102
103 # test accuracy
104
105 test_predictions = []
106 for i in range(Phi_test.shape[0]):
107     x = np.reshape(Phi_test[i], newshape: (p, 1))
108     pred = f(x)[0, 0]
109     test_predictions.append(pred)
110
111 test_predictions = np.array(test_predictions)
112
113 # RMSE
114 rmse = np.sqrt(np.mean((test_predictions - y_test) ** 2))
115 print(f"Test RMSE: {rmse:.4f}")
116
117 # MAE
118 mae = np.mean(np.abs(test_predictions - y_test))
119 print(f"Test MAE: {mae:.4f}")
120
121 # R-squared
122 ss_res = np.sum((y_test - test_predictions) ** 2)
123 ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
124 r2 = 1 - ss_res / ss_tot
125 print(f"Test R-squared: {r2:.4f}")
126
127 # store traditional metrics
128 rmse_trad = rmse
129 mae_trad = mae
130 r2_trad = r2
131
132 # load enriched data
133 data_enrich = pd.read_csv('data/movies_clean_enriched.csv')
134 enriched_cols = feature_cols + ['google_trends_interest']
135
136 Phi_e = data_enrich[enriched_cols].to_numpy()
137 y_e = data_enrich['log_revenue'].to_numpy()
138
139 scaler_e = MinMaxScaler()
140 Phi_e_scale = scaler_e.fit_transform(Phi_e)
141
142 n_e = Phi_e_scale.shape[0]
143 Phi_e_scale = np.column_stack([np.ones(n_e), Phi_e_scale])
144
145 [Phi_e_train, Phi_e_test, y_e_train, y_e_test] = train_test_split(
146     *arrays: Phi_e_scale, y_e, test_size=0.2, random_state=42)
147
148 # train enriched MLP
149 X_e = Phi_e_train
150 p_e = X_e.shape[1]
151
152 W1_e = np.random.randn(p_e, q) * np.sqrt(2/p_e)
153 W2_e = np.random.randn(q, 1) * np.sqrt(2/q)
154
155 def f_e(x): # 4 usages ± bellafratantonio
156     h = relu(W1_e.T.dot(x))
157     return W2_e.T.dot(h)
158
159 errors_e = []
160 n_e_tr = X_e.shape[0]

```

```

161
162 for epoch in range(epochs):
163     dW2_e = 0
164     for i in range(n_e_tr):
165         x = np.reshape(X_e[i], newshape: (p_e, 1))
166         h = relu(W1_e.T.dot(x))
167         dW2_e += (1/n_e_tr) * (f_e(x) - y_e_train[i]) * h
168     W2_e = W2_e - eta * dW2_e
169
170     dW1_e = 0
171     for i in range(n_e_tr):
172         x = np.reshape(X_e[i], newshape: (p_e, 1))
173         h = relu(W1_e.T.dot(x))
174         mat1 = np.heaviside(h, 0)
175         dW1_e += (1/n_e_tr) * (f_e(x) - y_e_train[i]) * np.kron(x, (W2_e * mat1).T)
176     W1_e = W1_e - eta * dW1_e
177
178     loss_e = 0
179     for i in range(n_e_tr):
180         x = np.reshape(X_e[i], newshape: (p_e, 1))
181         loss_e += (f_e(x)[0, 0] - y_e_train[i]) ** 2
182     errors_e.append(loss_e / n_e_tr)
183
184     if (epoch + 1) % 100 == 0:
185         print(f"Epoch {epoch+1}/{epochs}, MSE: {errors_e[-1]:.4f}")
186
187 # enriched test metrics
188 preds_e = np.array([f_e(np.reshape(Phi_e_test[i], newshape: (p_e, 1)))][0, 0]
189                    for i in range(Phi_e_test.shape[0])])
190
191 rmse_enrich = np.sqrt(np.mean((preds_e - y_e_test) ** 2))
192 mae_enrich = np.mean(np.abs(preds_e - y_e_test))
193 r2_enrich = 1 - np.sum((y_e_test - preds_e) ** 2) / np.sum((y_e_test - np.mean(y_e_test)) ** 2)
194
195 # comparison
196 print("\n--- traditional vs enriched ---")
197 print(f"RMSE: {rmse_trad:.4f} -> {rmse_enrich:.4f}")
198 print(f"MAE: {mae_trad:.4f} -> {mae_enrich:.4f}")
199 print(f"R2: {r2_trad:.4f} -> {r2_enrich:.4f}")

```

B. Bayesian MCMC Implementation (R)

```

16 Phi <- as.matrix(movies_data[, feature_cols])
17 y <- movies_data$log_revenue
18 Phi_scaled <- scale(Phi)
19
20 # add bias column
21 Phi_scaled <- cbind(bias = 1, Phi_scaled)
22
23 # train/test split
24
25 set.seed(42)
26 train_indices <- sample(1:nrow(Phi_scaled), size = round(nrow(Phi_scaled) * 0.8), replace = FALSE)
27
28 Phi_train <- Phi_scaled[train_indices, ]
29 Phi_test <- Phi_scaled[-train_indices, ]
30 y_train <- y[train_indices]
31 y_test <- y[-train_indices]
32
33 cat("Train:", nrow(Phi_train), "Test:", nrow(Phi_test), "\n")
34
35
36 log_likelihood <- function(y, X, beta, sigma2) {
37   n <- length(y)
38   resid <- y - X %*% beta
39   -n/2 * log(2 * pi * sigma2) - sum(resid^2) / (2 * sigma2)
40 }
41
42 log_prior_beta <- function(beta, tau2 = 100) {
43   p <- length(beta)
44   -p/2 * log(2 * pi * tau2) - sum(beta^2) / (2 * tau2)
45 }

```

```

47 ~ log_prior_sigma2 <- function(sigma2, a0 = 2, b0 = 1) {
48   if (sigma2 <= 0) return(-Inf)
49   a0 * log(b0) - lgamma(a0) - (a0 + 1) * log(sigma2) - b0 / sigma2
50 ~ }
51
52 ~ log_posterior <- function(y, X, beta, sigma2) {
53   log_likelihood(y, X, beta, sigma2) +
54   log_prior_beta(beta) +
55   log_prior_sigma2(sigma2)
56 ~ }
57
58
59 run_mcmc <- function(y, X, n_iter = 10000, burn_in = 2000,
60 ~   proposal_sd_beta = 0.02, proposal_sd_sigma2 = 0.1) {
61
62   p <- ncol(X)
63
64   # initialize
65   beta_cur <- as.numeric(solve(t(X) %>% X + diag(0.01, p)) %>% t(X) %>% y)
66   resid <- y - X %>% beta_cur
67   sigma2_cur <- as.numeric(sum(resid^2) / (length(y) - p))
68
69   # storage
70   beta_samples <- matrix(NA, n_iter, p)
71   sigma2_samples <- numeric(n_iter)
72
73   lp_cur <- log_posterior(y, X, beta_cur, sigma2_cur)
74
75 ~ for (i in 1:n_iter) {
76   # propose new beta
77   beta_prop <- beta_cur + rnorm(p, 0, proposal_sd_beta)
78   lp_prop <- log_posterior(y, X, beta_prop, sigma2_cur)
79 ~   if (log(runif(1)) < lp_prop - lp_cur) {
80     beta_cur <- beta_prop
81     lp_cur <- lp_prop
82 ~   }
83
84   # propose new sigma2
85   ls_cur <- log(sigma2_cur)
86   ls_prop <- ls_cur + rnorm(1, 0, proposal_sd_sigma2)
87   s2_prop <- exp(ls_prop)
88   lp_prop <- log_posterior(y, X, beta_cur, s2_prop)
89 ~   if (log(runif(1)) < (lp_prop + ls_prop) - (lp_cur + ls_cur)) {
90     sigma2_cur <- s2_prop
91     lp_cur <- lp_prop
92 ~   }
93
94   beta_samples[i, ] <- beta_cur
95   sigma2_samples[i] <- sigma2_cur
96
97   if (i %% 2000 == 0) cat(sprintf(" Iteration %d/%d\n", i, n_iter))
98 ~ }
99
100 keep <- (burn_in + 1):n_iter
101 list(beta = beta_samples[keep, ], sigma2 = sigma2_samples[keep])
102 ~ }
103
104 # run mcmc on traditional features
105 cat("Running MCMC traditional features...\n")
106 mcmc_trad <- run_mcmc(y_train, Phi_train)
107
108 # check convergence with trace plots
109 plot(mcmc_trad$beta[, 1], type = "l", main = "Convergence of beta bias", xlab = "Iteration", ylab = "beta")
110 plot(mcmc_trad$beta[, 2], type = "l", main = "Convergence of beta log_budget", xlab = "Iteration", ylab = "beta")
111 plot(mcmc_trad$sigma2, type = "l", main = "Convergence of sigma2", xlab = "Iteration", ylab = "sigma2")
112
113 # check ACF plots
114 acf(mcmc_trad$beta[, 1], main = "ACF beta bias")
115 acf(mcmc_trad$beta[, 2], main = "ACF beta log_budget")
116 acf(mcmc_trad$sigma2, main = "ACF sigma2")
117
118 # thin every 10th sample
119 thinned_beta <- mcmc_trad$beta[seq(1, nrow(mcmc_trad$beta), by = 10), ]
120 thinned_sigma2 <- mcmc_trad$sigma2[seq(1, length(mcmc_trad$sigma2), by = 10)]
121
122 acf(thinned_beta[, 1], main = "ACF beta bias thinned")
123 acf(thinned_beta[, 2], main = "ACF beta log_budget thinned")
124
125 # posterior coefficient summary
126 coef_names <- c("bias", feature_cols)
127 beta_means <- colMeans(thinned_beta)
128 beta_ci <- apply(thinned_beta, 2, quantile, c(0.025, 0.975))
129
130 coef_summary <- data.frame(
131   feature = coef_names,
132   mean = beta_means,
133   lower95 = beta_ci[, 1, ],
134   upper95 = beta_ci[, 2, ]
135 )
136 print(coef_summary)

```

```

138 # posterior predictive mean for each test point
139 pred_matrix <- Phi_test %>% t(thinned_beta)
140 y_hat_trad <- rowMeans(pred_matrix)
141
142 # 95% credible intervals
143 S <- nrow(thinned_beta)
144 pred_draws <- matrix(NA, nrow(Phi_test), S)
145 for (s in 1:S) {
146   mu_s <- Phi_test %>% thinned_beta[s, ]
147   pred_draws[, s] <- rnorm(nrow(Phi_test), mean = mu_s, sd = sqrt(thinned_sigma2[s]))
148 }
149 ci_lower <- apply(pred_draws, 1, quantile, 0.025)
150 ci_upper <- apply(pred_draws, 1, quantile, 0.975)
151 coverage <- mean(y_test >= ci_lower & y_test <= ci_upper)
152
153 # metrics
154 rmse_trad <- sqrt(mean((y_hat_trad - y_test)^2))
155 mae_trad <- mean(abs(y_hat_trad - y_test))
156 r2_trad <- 1 - sum((y_test - y_hat_trad)^2) / sum((y_test - mean(y_test))^2)
157
158 cat(sprintf("RMSE: %.4f\n", rmse_trad))
159 cat(sprintf("MAE: %.4f\n", mae_trad))
160 cat(sprintf("R2: %.4f\n", r2_trad))
161 cat(sprintf("95pct CI coverage: %.1f%%\n", coverage * 100))
162
163 # predicted vs actual
164 plot(y_test, y_hat_trad,
165      xlab = "actual log revenue", ylab = "predicted log revenue",
166      main = "Bayesian traditional predicted vs actual",
167      pch = 16, col = rgb(0, 0, 1, 0.4))
168 abline(0, 1, col = "red", lwd = 2)

```

```

170 # run mcmc on enriched features
171 data_enrich <- read.csv("data/movies_clean_enriched.csv")
172 enriched_cols <- c(feature_cols, "google_trends_interest")
173
174 Phi_e <- as.matrix(data_enrich[, enriched_cols])
175 Phi_e_scaled <- cbind(bias = 1, scale(Phi_e))
176 Phi_e_train <- Phi_e_scaled[train_indices, ]
177 Phi_e_test <- Phi_e_scaled[-train_indices, ]
178
179 cat("Running MCMC enriched features...\n")
180 mcmc_enrich <- run_mcmc(y_train, Phi_e_train)
181 thinned_beta_e <- mcmc_enrich$beta[seq(1, nrow(mcmc_enrich$beta), by = 10), ]
182 thinned_sigma2_e <- mcmc_enrich$sigma2[seq(1, length(mcmc_enrich$sigma2), by = 10)]
183
184 y_hat_enrich <- rowMeans(Phi_e_test %>% t(thinned_beta_e))
185 rmse_enrich <- sqrt(mean((y_hat_enrich - y_test)^2))
186 mae_enrich <- mean(abs(y_hat_enrich - y_test))
187 r2_enrich <- 1 - sum((y_test - y_hat_enrich)^2) / sum((y_test - mean(y_test))^2)
188
189 cat(sprintf("RMSE: %.4f\n", rmse_enrich))
190 cat(sprintf("MAE: %.4f\n", mae_enrich))
191 cat(sprintf("R2: %.4f\n", r2_enrich))
192
193 # comparison table
194 cat("\n--- traditional vs enriched ---\n")
195 cat(sprintf("RMSE: %.4f -> %.4f\n", rmse_trad, rmse_enrich))
196 cat(sprintf("MAE: %.4f -> %.4f\n", mae_trad, mae_enrich))
197 cat(sprintf("R2: %.4f -> %.4f\n", r2_trad, r2_enrich))
198

```

Bibliography

- AMW World Group. 2026. "Film & Box Office Statistics 2026 | Revenue & Streaming Data." AMW World Group.
<https://amworldgroup.com/statistics/film-box-office-statistics>.
- Bogaert, Matthias, Michel Ballings, Dirk Van den Poel, and Asil Oztekin. 2021. "Box office sales and social media: A cross-platform comparison of predictive ability and mechanisms." *Elsevier* 147 (August).
<https://doi.org/10.1016/j.dss.2021.113517>.
- Hancock, David, Claire Rousseau, Jeff Slee, Sacha Wunsch, and Vanessa Behrens. 2025. "Global Film Production Hits Historic High, Surpassing Pre-Pandemic Levels." WIPO.
<https://www.wipo.int/en/web/global-innovation-index/w/blogs/2025/global-film-production>.
- Hannan, Brian. 2022. "When Box Office Stats Were Born - 1969." The Magnificent60s.
<https://themagnificent60s.com/2022/01/12/when-box-office-stats-were-born/>.
- Kesharwani, Abhishek, and Rakesh Bharti. 2017. "Movie Rating Prediction Based on Twitter Sentiment Analysis." *Journal of Advanced Computing and Communication Technologies* 5, no. 1 (February): 5.
[https://www.researchgate.net/publication/392798597 MOVIE RATING PREDICTION BASED ON TWITTER SENTIMENT ANALYSIS](https://www.researchgate.net/publication/392798597_MOVIE_RATING_PREDICTION_BASED_ON_TWITTER_SENTIMENT_ANALYSIS).
- Lee, Sangjae, Bikash KC, and Joon Yeon Choeh. 2020. "Comparing performance of ensemble methods in predicting movie box office revenue." *Heliyon* 6, no. 6 (June). <https://doi.org/10.1016/j.heliyon.2020.e04260>.
- Li, Dawei, and Zhi-Ping Liu. 2022. "Predicting Box-Office Markets with Machine Learning Methods." *Entropy* 24, no. 5 (May): 711. 10.3390/e24050711.
- Tang, Shenglan. 2024. "The box office prediction model based on the optimized XGBoost algorithm in the context of film marketing and distribution." Plos One. <https://doi.org/10.1371/journal.pone.0309227>.
- Valliyot, Sahal. 2026. "Global Box Office Reaches \$30.5B in 2024, Projected to Reach \$33B in 2025." Motivate Val Morgan.
<https://motivatevalmorgan.com/2025/01/06/global-box-office-reaches-30-5b-in-2024-projected-to-reach-33b-in-2025/>.

I. Introduction and Literature Review (2 pages)

- Recognition and statement of the problem(s)
- A refined literature review

II. ML Methodology and Data Collection (2 pages)

- Discuss how the data were chosen, collected, and curated to match the problem(s) of interest
- Provide a brief overview of the two/three ML methods used and outline any adjustments made outside of the course content
 - * E.g.: if you use a hybrid collaborative filtering approach (for example which combines content-based and user-user), discuss how this works mathematically

III. ML Results and Interpretation (3 pages)

- Provide the major results of each of your methods, reporting things like predictive test error/accuracy, interpretation of coefficients, etc.
- Include no more than 4 plots/tables (you will want to critically assess which are most important for establishing your results), though these will not count towards your page limit

IV. Conclusions and Future Work (1 page)

- Summarize the findings of your study
- Be sure to discuss if all assumptions of your given methods have been met and if the analysis of the data was appropriate
- What drawbacks are there from analyzing data this way? What would you do differently if you were to continue working on the problem?

V. Appendix: Implementation (Does not count towards page count)

- Provide at the end of your report (in a labeled appendix) the python and R code for your implementations of the methods used in the project
- Only include the training implementation; no need to provide any data collection, preprocessing, testing or results work (those should be available in the GitHub repo)

- Recall that one of your methods must be implemented manually (without the use of pre-built packages except for standard linear algebra libraries, e.g. NumPy is of course okay, but Scikit-learn is not)